

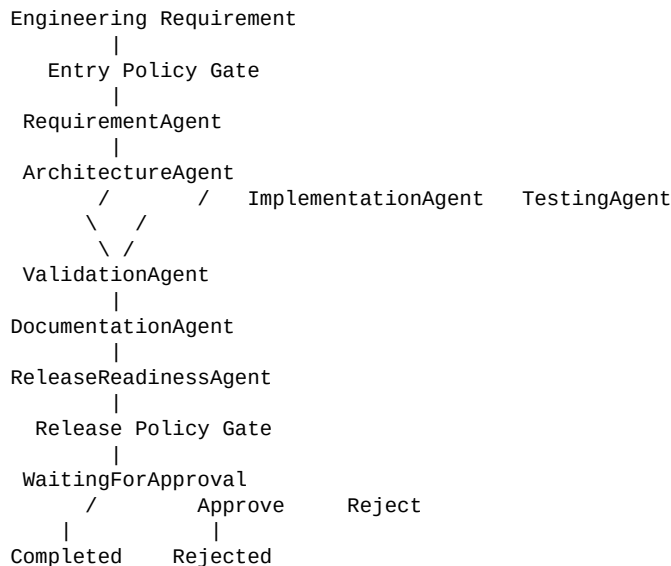
Agentic Software Engineering System

URL Shortener - Assessment Implementation and Learning Guide

Purpose: explain the complete prototype in beginner-friendly language while mapping the implementation to the supplied assessment.

1. What the system does

The application has two layers. First, a normal URL shortener creates short URLs, redirects them, and records analytics. Second, an agentic SDLC layer receives an engineering requirement and coordinates specialized agents through a governed workflow.



2. Why the orchestrator is the critical component

The orchestrator owns the process, not the specialized agents. It decides which graph nodes are ready, executes independent nodes in parallel, synchronizes dependent work, records context and audit events, applies retry/fallback/rollback rules, enforces policy gates, and stops at human approval.

This prevents the design from becoming a simple chain where one agent blindly calls the next.

3. Explicit dependency graph

Every WorkflowStep has an ID and DependsOn list. Implementation and test-design both depend on architecture, so they become ready together. The orchestrator executes all ready nodes with Task.WhenAll. Validation depends on both outputs, which creates an explicit synchronization point.

```
implementation.DependsOn = ["architecture"]
test-design.DependsOn    = ["architecture"]
validation.DependsOn     = ["implementation", "test-design"]
```

4. Entry and exit gates

Entry gate: validates that the requirement exists, the scenario is supported, and autonomous production deployment is not requested.

Release gate: checks that all automated stages completed and that ambiguous requirements preserve explicit assumptions. Passing the release gate still does not complete the run; human approval is required.

5. Context and decision lineage

WorkflowContext stores stage outputs, decisions, risks and assumptions. This gives later stages access to upstream reasoning. Dynamic replanning records parentRunId and the previous requirement so the new plan can be traced back to the original run.

6. Retry, fallback, rollback and safe-stop

Each agent gets at most two attempts. A retry is recorded in both the audit trail and reliability metrics.

If TestingAgent exhausts retries, the prototype uses a conservative fallback: mandatory manual validation and an explicit risk. If ImplementationAgent exhausts retries, there is no safe implementation fallback, so completed automated work is marked rolled back and the run stops.

Invalid requirements or blocked execution paths safe-stop rather than guessing.

7. Policy guardrails

PolicyGuardrail separates governance from agent reasoning. The demo blocks empty requirements, unsupported scenarios and autonomous production deployment. In production this boundary could call a policy-as-code service and include security, compliance and change-control rules.

8. Audit and observability

Every important transition creates an AuditEvent with timestamp, event type, message and optional step ID. The metrics endpoint exposes total runs, success rate, retries, rollbacks, replans, average latency and average recovery time.

9. Dynamic replanning

The replan endpoint demonstrates what happens when an upstream requirement changes. A closed run is not mutated. The prior run is superseded, a new governed run is created, planVersion increases, and parent/previous-requirement lineage is preserved.

10. Three required scenarios

Greenfield

Example: Add expiration dates to short URLs. Demonstrates new-feature decomposition, architecture, parallel implementation/test planning, synchronized validation and approval.

Brownfield

Example: Update existing analytics to include last access time. ArchitectureAgent maps the existing model, repository, service, controllers and tests, and records a smallest-compatible-change decision.

Ambiguous

Example: Make the URL shortener more secure. RequirementAgent identifies multiple possible interpretations, records assumptions/risks, and preserves uncertainty for human review.

11. Testing strategy

The UnitTests project verifies domain behavior and orchestration rules: graph dependencies, greenfield/brownfield/ambiguous behavior, entry policies, retries, fallback, rollback, approval and replanning lineage.

The IntegrationTests project uses WebApplicationFactory to exercise HTTP endpoints for health, URL creation, end-to-end workflow startup and metrics.

The TESTING_STEPS.md file contains a complete Swagger demonstration, including failure and recovery paths.

12. Setup

```
dotnet restore AgenticUrlShortener.sln
dotnet build AgenticUrlShortener.sln
dotnet test AgenticUrlShortener.sln
dotnet run --project src/AgenticUrlShortener.Api
```

Then open <https://localhost:7168/swagger>.

13. Important trade-offs

The prototype intentionally uses in-memory stores and deterministic C# agents. This makes it easy to run without external infrastructure or API keys and makes orchestration deterministic for testing. The trade-off is that it does not demonstrate real LLM reasoning or durable distributed state.

Rollback is a workflow-artifact rollback, not a Git/deployment rollback. Metrics are local to one process. The approval endpoint is not authenticated in the demo. These are explicitly documented limitations rather than hidden assumptions.

14. How to explain the design in an interview

A concise explanation: I separated agent responsibilities from orchestration governance. The orchestrator executes an explicit dependency graph, runs independent implementation and test-design work in parallel, synchronizes before validation, preserves context and decision lineage, applies bounded retries and safe fallback/rollback behavior, records audit and reliability metrics, supports dynamic replanning, and requires human approval before completion. The agents are deterministic adapters in this prototype so the orchestration is runnable and testable without external API keys; the same boundaries can later host real LLM/tool agents.

15. File map

```
AgenticUrlShortener.sln
src/AgenticUrlShortener.Api/
  Controllers/
  Models/
  Repositories/
```

```
Services/  
  Orchestration/  
tests/  
  AgenticUrlShortener.UnitTests/  
  AgenticUrlShortener.IntegrationTests/  
docs/  
  ARCHITECTURE.md  
  SCENARIOS.md  
  TESTING_STEPS.md  
  FINAL_ENGINEERING_SUMMARY.md  
  Agentic_AI_Assessment_Guide.pdf
```